

Препроцессоры CSS в практике веб-дизайна: возможности и перспективы

Беляев А.А., преподаватель кафедры фотожурналистики и технологий СМИ факультета журналистики МГУ им. М.В. Ломоносова, кандидат филологических наук.

certainlyakey@gmail.com

Статья посвящена роли препроцессоров CSS в современном веб-дизайне. Подробно рассматриваются возможности препроцессоров, их специфика, ограничения и значение для веб-дизайнеров.

Ключевые слова: веб-дизайн, CSS, препроцессоры, стиль, отзывчивый дизайн

The article discusses the role of CSS preprocessors such as SASS, LESS and Stylus in the modern web design. The preprocessors' features, limitations and implications for web design are examined.

Keywords: web design, CSS, preprocessors, style, responsive design

Язык стилового оформления CSS (Cascading Style Sheets) уникален в своем роде — у него нет конкурентов. В среде веб-разработки насчитывается много аналогов серверных языков программирования, разработчику есть из чего выбрать — PHP, .NET, Java и другие. В сфере визуального оформления веб-страниц ситуация иная. Каскадные таблицы стилей стали не имеющим аналогов инструментом как де-факто, так и де-юре, в качестве документа W3C¹. Это определило внимание и объем требований, которые дизайнеры и разработчики предъявляют к набору возможностей языка CSS.

Основная задача этой статьи — рассмотреть технологии, которые называют препроцессорами CSS. Препроцессоры задумывались как дополнение к CSS и призваны значительно расширить его возможности. Наша дополнительная задача — показать, каким образом препроцессоры CSS облегчают работу веб-дизайнера с точки зрения быстрой и корректной реализации дизайнерской идеи в веб-макете.

С точки зрения информатики препроцессоры CSS относятся к т.н. метаязыкам — языкам, являющимся надстройками над другими языками более низкого уровня. С точки зрения способа записи кода это выражается в том, что препроцессор может совпадать с языком низшего уровня по синтаксису, расширяя его, но не меняя принципиально. Название "препроцессор" означает, что исходный код метаязыка компилируется (преобразуется) в код на низшем, часто более распространенном языке. Именно так работают препроцессоры языка стилового оформления CSS. С помощью механизма препроцессинга удастся расширить возможности CSS, избежав проблемы, типичной для всех новых технологий, — плохой поддержки существующим программным окружением и устройствами. Особенно эта проблема актуальна для клиентских веб-технологий, ведь обычно до полной поддержки всеми клиентами пользователя (браузерами) проходит долгое время.

¹ Полная спецификация CSS 2.1 находится по адресу <http://www.w3.org/TR/CSS21/>.

Препроцессоры в реальном мире

На текущий момент в активной разработке находятся несколько препроцессоров CSS. SASS, по некоторым данным, самый популярный на текущий момент², был разработан Х. Кэтлин и Н. Вейзенбаумом в 2007 г., незадолго до этого создавшими препроцессор HTML — HAML. SASS расшифровывается как Syntactically Awesome Stylesheets ("синтаксически потрясающие таблицы стилей"). Конкурирующий продукт под названием LESS (Leaner CSS — "гибкий CSS") годом позже начал развивать А. Селлье³. Последний препроцессор в этом ряду, Stylus, разрабатывается Т. Головайчуком с 2010 г. (он автор также другого препроцессора для HTML, Jade). Все 3 препроцессора обладают похожим набором функций, различаясь только двумя вариантами синтаксиса. В первом варианте для простоты опущены фигурные скобки, разделяющие декларации, а начало нового правила обработчик кода вычисляет по присутствию отступа перед ним. Этот синтаксис был унаследован SASS у "старшего брата" HAML и позже распространился на Stylus. Вторая разновидность синтаксиса по умолчанию совместима с обычным CSS, используя фигурные скобки и не учитывая отступы при обработке кода. Препроцессор LESS, появившись год спустя после создания SASS, взял за основу синтаксиса именно обычную запись CSS. Спустя некоторое время и в самом SASS появилась возможность писать в совместимом с CSS синтаксисе (этот вариант был назван SCSS и получил формальный приоритет у разработчиков), но классический вариант с отступами остался (он именуется просто SASS и также полностью поддерживается). Запущенный последним, Stylus взял на вооружение минималистично интерпретированную запись с отступами, не запрещая, однако, фигурные скобки и другие обозначения из традиционного CSS. Таким образом, оба варианта синтаксиса достаточно распространены на сегодняшний день; кто-то предпочитает минималистичность старого варианта, а кому-то по душе низкий входной порог нового, так как уже написанный код CSS не нужно переписывать заново для включения в него функциональности, привносимой препроцессором⁴. Общая эволюция синтаксиса препроцессоров идет в направлении максимального уподобления синтаксису CSS, о чем свидетельствует развитие SASS, постепенно заменившего или удалившего все символы, напоминавшие о первоначальном стиле записи в стиле HAML. Рискнем предположить, что это и послужило причиной массового роста их популярности, в отличие от того же HAML⁵).

В частных примерах, которые встретятся читателю в этой статье, мы будем рассматривать синтаксис SCSS препроцессора SASS.

² Coyier C. Statistics on Preprocessor Usage // CodePen Blog. 2013. — 8 Apr. — <http://blog.codepen.io/2013/04/08/statistics-on-preprocessor-usage/>.

³ См. подробное сравнение активности двух проектов — Coyier C. Sass vs. LESS // CSS-Tricks. 2012. — 16 May. — <http://css-tricks.com/sass-vs-less/>.

⁴ См. подробнее Long J.W. Sass vs. SCSS: Which Syntax is Better? // The Sass Way. 2011. — 12 Sept. — <http://thesassway.com/articles/sass-vs-scss-which-syntax-is-better>.

⁵ Cooper M. Hampton Catlin on building Sass // .net Magazine. 2013. — 18 Feb. — <http://www.netmagazine.com/interviews/hampton-catlin-building-sass>.

Главной мотивацией для создания SASS его создателем Х. Кэтлин⁶ было названо недовольство вышедшей в 1998 г. версией CSS 2.1⁶. Кэтлин, в частности, упоминает, что, по его мнению, утвержденному стандарту не хватало поддержки переменных, вложенности правил и манипуляций с цветом. Кэтлин не единственный из веб-дизайнеров не был удовлетворен функциональностью опубликованной W3C рекомендации, однако тогда, как и сейчас, большой приоритет для разработчиков CSS имела необходимость реализации приемов визуального оформления, позволявших приблизиться к, например, заданному настольными издательскими системами стандарту. В этом плане ситуация выравнивалась только с появлением полностью поддерживающих стандарт CSS2.1 браузеров, а вслед за ними модулей CSS3 во второй половине 2000-х: веб-дизайнеры наконец перестали ограничивать себя в творческой свободе, а на веб-страницы обрушился шквал генерируемых при помощи таблиц стилей приемов и эффектов: скругленных уголков, теней, градиентов, несистемных шрифтов и т.д.

Ограничения и недостатки CSS

Вместе с тем с развитием CSS появились и "проблемы роста". Постараемся здесь их перечислить.

Проблемы с кроссбраузерностью функций CSS

Имплементация новшеств CSS3 зачастую не синхронизирована у разных браузеров и тем более идет в более быстром темпе, чем их формальное утверждение Консорциумом Всемирной паутины (W3C). На практике это способствовало усилению системы префиксов, когда поддержка еще формально не утвержденного свойства CSS появляется в браузере — но с префиксом, дабы не конфликтовать с разнящейся имплементацией в другом браузере. В результате, если свойство и привнесенная им функциональность пользуются популярностью, приобретает распространение тройная или более форма его записи сразу для всех браузеров по схеме:

```
-webkit-<свойство>: значение;  
-moz-<свойство>: значение;  
-o-<свойство>: значение;
```

Следствием этого становится излишнее раздутие кода CSS и повышение его субъективной сложности для разработчика. Похожая ситуация сложилась с отсутствием единой поддержки определенных форматов ресурсов, подгружаемых CSS. Так, правило `font-face`, отвечающее за подгрузку нестандартных шрифтов на страницу, на данный момент требует указания минимум 3 файлов шрифта в различных форматах.

⁶ Там же.

Повтор деклараций и увеличение объема CSS

По мере того, как "взрослеет" оформительский арсенал CSS, растет и объем правил оформления в нем, а с ним и средний объем файла. Основная причина — постепенная трансформация все большего числа средств оформления в "нативный" CSS. Так, свойство `border-radius` заменило традиционный подход к скруглению уголков при помощи изображений, то же происходит в настоящее время с градиентами. Кое-какая новая функциональность требует подробного изложения в коде в силу своей природы. Например, это относится к тем же градиентам, при наличии в которых нескольких точек перехода декларация сильно разрастается. Новые, более удобные способы указания цвета (`rgba`, `hsla`) занимают и больше места, чем традиционное указание при помощи шестнадцатичного числа. Кроме того, распространение среди веб-дизайнеров семантического подхода к верстке повлекло за собой исключение некоторых несемантических элементов из HTML — и перенос их в таблицы стилей в виде псевдоэлементов и псевдоклассов.

Однако это далеко не все факторы, влияющие на рост объема кода CSS. Растет объем кода, зеркально повторяющегося в нескольких местах. Это объясняется несколькими причинами. Так, шаблонный характер веб-дизайна и единая графическая модель как фундамент макета веб-страницы обуславливают внешнее подобие элементов в разных ее частях, а стало быть, повтор одних и тех же визуальных средств. Также код может дублироваться из-за необходимости включать в код разных блоков один и тот же технический прием, позволяющий обойти какие-то недостатки CSS. Например, всевозможные методы растягивания блока с плавающими элементами по высоте контента (англ. *clearing floats*) требуют прописывания дополнительных свойств или, сверх того, добавления селекторов псевдоэлементов `:before` и `:after`.

В CSS существует встроенная возможность группировать селекторы через запятую, указывая декларации лишь единожды. Однако в этом случае оптимизация кода становится отдельным этапом, который вынуждает разработчика менять его структуру и разрушает обычную группировку по элементам и комплексам на странице.

Частый повтор одних свойств оформления чреват не только увеличением объема кода, но также повышенной вероятностью ошибки или отклонения в оформлении страницы, особенно у свойств типа `box-shadow`, предполагающих составные значения.

Повторяться могут не только декларации, но и селекторы. Эта проблема, как правило, проявляется при активном использовании селекторов потомков, когда к разным элементам внутри одного и того же контейнера обращаются через его контекст, указывая в селекторе родителя. Например:

```
.column .anons h1 {...}  
.column .anons .links {...}
```

```
.column .anons .desc {...}
```

Неудивительно, что, по данным статистического ресурса HTTP Archive, за 2,5 года⁷ средний объем CSS-файла возрос в 1.5 раза, с 25 Кб до 37 Кб.

CSS как язык для верстки, а не дизайна

В большинстве веб-проектов перед началом этапа верстки страниц предполагается, что уже создан графический макет, которому верстка во всем следует. Этой же логике следует и CSS как веб-стандарт, предоставляя оформительские средства в большинстве своем в том объеме, какой требуется для воплощения в коде уже существующего, детализированного макета. Так, CSS до недавнего времени не давал в руки дизайнера интуитивных средств для манипуляций цветом в самом коде: так, при необходимости осветлить или затемнить какой-либо цвет дизайнеру приходилось пользоваться сторонними программами, например, Adobe Photoshop для того, чтобы определить конечный цвет. Отчасти эту проблему призвана решить цветовая модель HSL, вошедшая в арсенал каскадных таблиц стилей не так давно: дизайнер получает прозрачный инструмент, позволяющий отдельно управлять оттенком, насыщенностью и яркостью цвета. Однако HSL еще не обладает той мерой поддержки браузеров и устройств, которая обычно считается достаточной для вхождения в повседневный обиход веб-дизайна. Кроме того, в CSS все связанные с цветом свойства выражаются через конкретные значения цветов, что не дает возможности быстро определить отношения — светлее, темнее, ярче или насыщенней — между ними.

То же касается и традиционной для графического и веб-дизайна пропорциональности размеров элементов и белого пространства. В графических редакторах и НИС эта проблема решается возможностью проводить базовые вычисления прямо в процессе задания размеров и расстояний между блоками. В CSS можно лишь напрямую выставлять нужные значения в числах. Единицы измерения `em` отчасти позволяют установить явную систему взаимоотношений между размерностями в коде, но в случае с пикселями это уже невозможно.

Все эти проблемы попытались решить разработчики препроцессоров. Как именно — мы увидим ниже.

Механизм подключения и работы

Алгоритм подключения к проекту у разных препроцессоров разный и зависит от веб-технологии, на которой основывается код препроцессора. У SASS это Ruby, у LESS JavaScript/node.js и Stylus — node.js. Однако в общем виде работа с препроцессором требует наличия установленного программного окружения данной

⁷ См. Interesting stats // HTTP Archive. 2013. — 16 June. — <http://httparchive.org/interesting.php>. Статистика ведется с ноября 2010 г.

технологии на сервере или локальной машине разработчика. Такая платформа уже имеется у тех, кто разрабатывает сайты на node.js или Ruby, но для остальных установка ее сопряжена с различными трудностями и выходит за рамки начальных знаний по веб-разработке, что, безусловно, повышает входной порог препроцессоров CSS в общем. Своеобразное исключение составляет LESS, основанный на JavaScript, — он способен работать прямо в браузере, следовательно, для начала работы достаточно подключить к HTML-странице стили препроцессора и его скрипт. Однако такой вариант не подходит для готового сайта, поскольку дополнительно нагружает браузер пользователя со стороны как отрисовки страницы, так и скорости загрузки. SASS и Stylus не располагают по умолчанию такой возможностью. Преобразование кода препроцессора, находящегося в условном файле `style.scss`, в код CSS-файла `style.css` производится в ручном или полуавтоматическом режиме с помощью запуска небольшой программы в командной строке.

Существуют также специальные приложения с графическим интерфейсом наподобие LiveReload или Codekit, как правило, простые в установке, которые автоматически преобразуют код препроцессора в код CSS при сохранении первого. Все необходимые для запуска препроцессора библиотеки и исполняемые файлы они уже содержат, что снижает порог вхождения, связанный со сложностью установки, практически до нуля. Тем не менее такие приложения зачастую платные, поэтому ситуацию нельзя назвать полностью удовлетворительной.

Основные функции препроцессоров CSS

Препроцессоры призваны повысить качество работы с CSS по тем направлениям, которые мы указали выше, а также — заодно — по мере возможности облегчать жизнь веб-разработчикам различными дополнительными функциями. Рассмотрим здесь основные функции, которые реализованы в популярных препроцессорах таблиц стилей.

Переменные

Препроцессоры вводят в инструментарий стилового оформления понятие переменных, давно знакомое тем, кто работает с языками программирования. В ракурсе дизайна корректной будет аналогия с сохраненными образцами цветов (swatches) в программах верстки печатных изданий и графических редакторах. В переменной можно "сохранить" значение или набор значений:

```
$alert_color: red;  
$headings_font: Arial, Helvetica, sans-serif;  
$gallery_item_width: 250px;
```

В дальнейшем переменную, название которой всегда начинается на символ \$, можно использовать в любом месте документа CSS. Основное преимущество

переменных — легкость внесения глобальных изменений в таблицу стилей. Переменные также ощутимо разгружают код в случае, если в них сохраняются достаточно объемные значения свойств, например, `font` или `background`. Наконец, подобно селекторам классов и `id`, переменные несут семантическую нагрузку, облегчающую чтение и улучшающую логическую организацию стилевого документа: `$alert_color` с субъективным смыслом "предупреждающий цвет" проще идентифицировать, нежели цветное значение вида `#ff0000`. Как и в случае с селекторами, это требует соответствия наименования переменной критерию семантической; переменная должна быть названа так, чтобы при редизайне сайта сохранить свою смысловую нагрузку⁸. Как зачастую отмечается⁹, список корректно именованных переменных может послужить своеобразным аналогом "гида по стилю" (англ. *style guide*) веб-сайта.

Отметим, что концепция переменных в общих чертах соответствует изначальным задачам CSS, а также принципу единообразия оформления. Переменные, по сути, продолжают идею, на которой базируются классы и `id`. Использование переменных способствует формированию единой визуальной организации веб-страницы, поскольку форсирует приведение разнящихся характеристик оформления к единому знаменателю.

Миксины

Другое понятие в лексиконе препроцессоров, напоминающее о языках программирования, — это миксины (от англ. *mix in*, "примешивать"). Основная задача, которую выполняет миксин, — это также хранение некоторых данных для повторного использования. По сравнению с переменными миксины расширяют спектр данных: в них допустимо хранение уже целых деклараций и правил CSS вместе с селекторами. Определение миксина в SASS происходит при помощи ключевого слова `@mixin`, включение в код — ключевым словом `@include`, после которого следует название миксина.

```
@mixin setliststyling {  
    ul li {list-style-type:disc; list-style-  
position:inside;}  
}
```

Помимо характера хранимых данных, миксины отличаются от переменных тем, что допускают наличие аргументов — произвольных данных, которые позволяют частично менять состав миксина на этапе его включения в код. Аргументы

⁸ См. подробнее о принципах именования классов Wilcox M. Naming and using id's and classes properly // Matt Wilcox Blog. 2011. — 25 July. — <http://2002-2012.mattwilcox.net/archive/entry/id/1058/>.

⁹ Bolton J. Front-end Maintainability with Sass and Style Guides // Engine Yard Developer Blog. 2011. — 19 Jan. — <https://blog.engineyard.com/2011/front-end-maintainability-with-sass-and-style-guides>.

указываются в определении миксина в скобках после его названия. Идея аргументов позаимствована у функций в языках программирования. Так же, как и в функциях, возможно указание аргумента по умолчанию, что еще более сокращает код.

```
@mixin pixel-shadow($shadow_color, $border_color:black) {  
  box-shadow:0px 1px 0px $shadow_color;  
  border-bottom:1px solid $border_color;  
}
```

```
@include pixel-shadow(#ccc);
```

В практике веб-дизайна миксины позволяют добиться следующих целей:

- становится возможным типизировать запись повторяющихся в разных местах страницы характеристик оформления и их наборов. В традиционном CSS свойства или правила приходится каждый раз дублировать целиком, что чревато тяжеловесностью кода и связанной с ней пониженной читаемостью, ошибками в синтаксисе и риском утраты единообразия. В практике дизайна это особенно актуально в силу популярности определенных комплексных приемов (пиксельных теней, "бабблов", круглых элементов и т.д.), которые требуют объемной записи и/или привлечения дополнительных селекторов;
- заметно уменьшается объем кода, требующегося для кроссбраузерной записи свойств с префиксами или, к примеру, указания нестандартных шрифтов при помощи правила `@font-face`¹⁰;
- благодаря аргументам становится возможным малым количеством кода варьировать дизайн, сохраняя читаемость;
- миксины позволяют включать вариации оформления для разных условий медиазапросов прямо внутри селектора оформляемого элемента, делая более удобным и логичным написание кода для отзывчивых макетов¹¹.

Миксины разрешено вкладывать друг в друга; это означает, к примеру, что их можно организовать в виде двух уровней: технического (куда будут входить переходящие из проекта в проект миксины, например, префиксные) и оформительского (он будет служить главным образом для унификации приемов оформления или даже дизайна целых блоков и может содержать "технические" миксины).

¹⁰ Правда, существуют свидетельства, что развертывание префиксных свойств при помощи встроенных миксинов осуществляется препроцессорами зачастую некорректно. См. презентацию В. Макеева «-Пре-фиксы. Зачем и как правильно» по адресу <http://pepelsbey.net/pres/pre-fixes/> (слайды 25-34).

¹¹ См., например: Richard S. Responsive Web Design with Sass+Compass // Snugug.github.io. – <http://snugug.github.io/RWD-with-Sass-Compass/>.

Недостаток миксинов, от которого можно избавиться с помощью аргументов, но который тем не менее может встречаться достаточно часто — это их низкая гибкость. Так как оформительские миксины, как правило, создаются под конкретную визуальную идею, в миксине могут оказаться те свойства, которые в определенных условиях могут быть лишними или даже стать причиной ошибки. В этом плане отладка миксина, непрозрачного по своей природе, будет также затруднительна, а исправление ошибки может потребовать переформулирования миксина или вовсе отказа от него. Эта проблема является общей для любых средств унификации оформления как в веб-дизайне, так и в дизайне графическом. Кроме того, чрезмерное количество миксинов может сильно раздуть код, из-за чего некоторые авторы¹² рекомендуют отказываться от их использования без аргументов. Из этого следует, что планирование структуры миксинов следует проводить тщательно и на этапе, предшествующем непосредственно верстке.

Директива `@extend`

Использование миксинов снижает объем файла препроцессора, однако может отрицательно повлиять на объем скачиваемого пользователями файла CSS. На минимизацию объема конечного кода направлена директива `@extend`, на данный момент реализованная в той или иной форме во всех трех популярных препроцессорах.

`@extend` позволяет включить все оформление, присвоенное какому-либо селектору, в другое правило:

```
.button {padding:20px;}
.form-button {@extend .button; font-size:24px;}
```

При этом в конечном коде происходит следующее — селектор, в котором использована директива `@extend`, группируется с "копируемым" селектором:

```
.button, .form-button {padding:20px;}
.form-button {font-size:24px;}
```

Группирование селекторов, применяющих одни и те же декларации, дает возможность уменьшить общее количество кода. Директива `@extend`, таким образом, находится в полном соответствии с принципом DRY (англ. *Don't Repeat Yourself* — "Не повторяйся"), принятым в программировании, — в отличие от миксинов. Он часто используется для реализации чисто технических приемов наподобие растягивания блока с плавающими элементами по высоте контента:

```
.clearfix:before, .clearfix:after {content:"";
display:table;}
```

¹² Tomeij R. Sass doesn't create bad code. Bad coders do. // The Sass Way. 2012. — 2 Feb. — <http://thesassway.com/articles/sass-doesnt-create-bad-code-bad-coders-do>.

```
.clearfix:after {clear:both;}

.navigation {@extend .clearfix;}
```

@extend позволяет без особого вреда для размера HTML- и CSS-документов реализовать многоуровневую систему наследуемого оформления, подобную той, которая присутствует в некоторых программах "бумажной" верстки. В такой системе элемент может оформляться при участии базового стиля и дополнительных, уточняющих или изменяющих его. В классической клиентской веб-разработке это можно сделать, введя дополнительные названия классов в атрибут HTML class — в таком случае оформление обоих классов суммируется. Однако это может увеличить объем уже HTML-файла, затруднить его редактирование и ухудшить семантические качества. В случае использования @extend в HTML необходим уже только один класс, в котором содержимое всех селекторов уже объединено препроцессором.

SASS также вводит т.н. плэйсхолдеры (placeholders) — разновидность директивы @extend, не выводящая декларации копируемого селектора в конечном коде до тех пор, пока он не будет упомянут в другом селекторе. Это образует в CSS уровень абстракции, свойственный многим языкам программирования, и позволяет, например, создавать библиотеки стилей для реализации типовых макетов и приемов, которые появятся в коде CSS только тогда, когда они действительно нужны¹³. В целом, @extend и плейсхолдеры по новому освещают идеи, призванные минимизировать и упорядочить традиционный CSS, например, метод объектно-ориентированного CSS¹⁴ или стили написания CSS, основанные на концепции "модульного" оформления (БЭМ компании "Яндекс"¹⁵ или SMACSS¹⁶).

Вложенность селекторов

Чуть выше мы говорили об одной из проблем классического CSS — чрезмерной длине селекторов элементов, находящихся внутри одного родителя. Препроцессоры имеют специальное средство для борьбы с ними: вложенность селекторов.

```
.left_column {
    width:400px;
    h1 {font-size:24px;}
    .desc {padding-top:10px;}
```

¹³ Taylor I. OOCSS + Sass = The best way to CSS // Ian Storm Taylor Blog. 2012. – October. – <http://ianstormtaylor.com/oocss-plus-sass-is-the-best-way-to-css/>.

¹⁴ Lazaris L. An Introduction To Object Oriented CSS (OOCSS) // Smashing Magazine. 2011. – 12 Dec. – <http://coding.smashingmagazine.com/2011/12/12/an-introduction-to-object-oriented-css-oocss/>.

¹⁵ См. <http://ru.bem.info/>.

¹⁶ См. <http://smacss.com/>.

}

Так как концепция вложенности является фундаментальной для многих веб-технологий и используется в том числе в самом CSS (правило `@media`), мы можем посчитать вложенность селекторов интуитивно понятной функцией. Учитывая последовательность процесса верстки, в практике веб-разработки правила обычно группируются по принадлежности к одному блоку-контейнеру. Вложенные селекторов выносят общий компонент всех селекторов (чаще всего класс или `id` контейнера) в начало группы. При надлежащем форматировании кода глубину вложенности элемента можно определить визуально, по длине отступа.

Разработчик может использовать вложенные селекторы не только для группирования оформления элементов, находящихся внутри одного контейнера, но и для записи вариантов оформления этого контейнера (реализуемых при помощи классов или псевдоклассов). Сюда входит оформление его внутренних элементов, меняющееся при смене класса контейнера или условий медиазапроса, а также оформление соседних и дочерних элементов (всех, которые требуют наличия символа-комбинатора в селекторе). Включить варианты оформления контейнера внутри его группы можно с помощью символа `&`, ссылающегося в селекторе на родительский элемент:

```
a {  
    color:black;  
    &:hover {color:green;}  
    &.external {color:blue;}  
}
```

Как видно, в одной группе можно записать сразу весь код, тем или иным образом относящийся к данному блоку-контейнеру и его содержимому. Множество вариантов группирования селекторов позволяет разработчикам гибко подстраивать стиль написания кода под себя. В то же время глубокая вложенность селекторов вкупе с большим количеством внутренних элементов может стать причиной проблем со специфичностью¹⁷, в первую очередь в силу радикального снижения читаемости конкретного селектора и, соответственно, сложностью определения его специфичности. Глубокая вложенность, поощряемая легкостью организации блоков при работе с препроцессором, также противоречит методикам "модульного" оформления¹⁸. Простота и внешняя логичность организации кода оказывается в конфликте с требованиями к размеру конечного CSS-документа и идеей повторного использования стилей. В этом плане препроцессоры не способствуют формированию у пользователей единого и гармоничного стиля записи CSS, поскольку разные их возможности стимулируют разные стили организации кода.

¹⁷ Coyier C. David Walsh on Redesigning with Sass // CSS-Tricks. 2012. – 17 Oct. – <http://css-tricks.com/redesigning-with-sass/>.

¹⁸ Long J.W. Avoid nested selectors for more modular CSS // The Sass Way. 2013. – 24 May. – <http://thesassway.com/intermediate/avoid-nested-selectors-for-more-modular-css>.

Манипуляции с цветом

Помимо средств организации кода, препроцессоры добавляют функции для разнообразных действий со значениями в CSS. Наиболее полно реализованы возможности по манипуляциям с цветом и численными значениями. Рассмотрим каждое из них в отдельности.

Операции с цветовыми значениями могут производиться как в арифметической форме (простыми вычислениями — сложением значений цветов, умножением и т.д.), так и при помощи различных встроенных функций, заметно упрощающих все действия. Большинство таких функций основаны на гибкой цветовой модели HSL, однако они способны принимать в качестве аргументов и значения в любых других способах записи. Так, функция `lighten()` принимает два аргумента — цвет и сдвиг по яркости в процентах, выдавая на выходе новое значение цвета. Среди других встроенных функций можно выделить `desaturate()` (уменьшение насыщенности), `transparentize()` (повышение прозрачности) и `mix()`, позволяющую смешивать цвета. Как правило, препроцессоры позволяют создавать и свои собственные функции на базе встроенных, что дает возможность, например, имитировать режимы наложения из графических редакторов¹⁹.

Подобно регулировке количества белого (*tint*) для цветовых образцов в программах верстки, запись цвета в виде функции с прозрачным названием облегчает визуальное прочтение кода и выстраивание иерархии отношений между цветами, которая базируется обычно на фундаменте из пары-тройки основных цветов и их дальнейшей нюансировке. Использование сохраненной в переменные палитры основных цветов и их модификация при помощи функций соответствует в целом принципам организации и варьирования цветов в фирменном стиле и графической модели сайта.

Операции с числами

CSS насыщен различными размерностями и, соответственно, численными значениями. Все вычисления при написании кода нужно проводить самостоятельно, указывая лишь готовое значение (в отличие от, например, программ издательской верстки, где простые арифметические операции можно осуществлять прямо в полях, допускающих цифровой ввод). Препроцессоры позволяют легко производить любые численные операции прямо в коде, записывая их в виде выражений; в конечный CSS-файл попадают уже готовые значения. Часто эти вычисления проводятся со значениями, уже сохраненными в переменные. Можно назвать несколько задач, решаемых таким образом:

- в коде становится возможным явно задавать пропорциональные отношения (ритм) между размерами блоков, высотой строк, кеглями шрифтов и т.д. Как

¹⁹ См. <https://github.com/cloudhead/less.js/pull/596>.

следствие, облегчается разработка сеток. Ниже мы остановимся на этом подробнее;

- вопросы, связанные с вычислениями компонентов блочной модели, можно решать прямо в коде явным образом. Так, задача вычитания отступов из визуальной ширины блоков, следующая из стандартной блочной модели, разрешается быстро при помощи выражения типа `width:$width - $padding * 2;`
- повышается гибкость и скорость работы с вариациями в оформлении, которые должны происходить при выполнении определенных условий, скажем, при наличии конкретного числа дочерних элементов в блоке;
- облегчается работа с размерностями вариантов макета в рамках "отзывчивого дизайна".

Численные операции в препроцессорах позволяют транслировать работу с визуальным ритмом в макете с этапа создания графического макета на этап верстки. Возможность сохранить базовый шаг или отношение в переменную для использования в дальнейшем при задании пропорциональных размерностей с помощью выражений вида `$module-step * 1.5` или `$base-font-size * 2` позволяет выстроить аккуратную и четкую визуальную иерархию. При этом контроль над общим множителем остается в руках дизайнера, и в случае нужды переменную, его содержащую, можно поменять или переопределить, масштабируя таким образом весь макет.

Препроцессоры также могут оказать значительную помощь при создании "отзывчивого макета" (методика "отзывчивого дизайна"). В частности, численные операции могут пригодиться при конвертации абсолютных единиц в относительные для создания резиновых макетов, основанных на пропорциях фиксированного макета по формуле

результат = целевой объект ÷ контекст

На выходе формулы мы получаем значение искомой размерности в em'ах или процентах²⁰. Упростить запись вычислений для последних могут помочь встроенные функции (например, функция `percentage` в SASS).

Таким образом облегчается работа с пропорциями, модульными сетками и сетками базовых линий. Особое значение это имеет в связи с тем фактом, что акцент в работе с сетками в веб-дизайне сегодня переносится с графического редактора на код — доказательством тому распространение различных CSS-библиотек (960 Grid System, Blueprint и др.), делающих имплементацию сеток более доступной и простой в настройке. Библиотека функций Compass, основанная на SASS, также

²⁰ Marcotte E. Responsive Web Design. A Book Apart, New York, 2011. P. 20.

предлагает тонкий контроль над сеткой колонок и вертикальным ритмом на странице²¹.

Вычисляемые значения — одна из немногих функций препроцессоров на данный момент, которая получила развитие в качестве модуля CSS3 — `calc()` — и поддерживается современными версиями популярных браузеров. Другое дело, что необходимость этого до сих пор оспаривается. Главный аргумент против — негативное воздействие `calc()` на скорость отрисовки страницы. При использовании вычислений в коде препроцессора все операции производятся до генерации конечного файла CSS, поэтому этой проблемы нет. С другой стороны, функция `calc()`, работая с уже выведенной в браузер страницей, может подстраиваться под изменяющиеся условия просмотра и, как следствие, производить операции между относительными и абсолютными единицами. Так, выражение `100% - 20px` будет корректным для `calc()`, но при генерации препроцессором файла CSS вызовет ошибку.

Другие функции

Среди менее значимых, но часто встречающихся в арсенале препроцессоров функций можно назвать:

- директиву `@import`, включающую файл препроцессора или CSS напрямую в главную таблицу стилей. В отличие от правила `@import` в классическом CSS, наличие этой директивы не означает лишнего HTTP-запроса к серверу, что благотворно влияет на скорость загрузки и развязывает разработчикам руки для создания файловой структуры стилей, отражающей, например, весь спектр вариантов отзывчивого дизайна;
- различные стили вывода конечного файла CSS — от сжатого, минимизирующего объем документа, до расширенного;
- внедрение изображений в файл CSS в формате `data URI`, позволяющее сократить количество HTTP-запросов к серверу;
- автоматическое создание "спрайтов" — объединенных в один файл изображений, что призвано также уменьшить количество HTTP-запросов;
- конкатенация строк позволяет сохранить в переменной, например, базовую часть адреса для упоминаемых в таблице стилей ресурсов;
- директивы условий и циклов и т.д.

Ограничения препроцессоров

В этой статье мы называли некоторые ограничения и недостатки препроцессоров. Ниже мы сведем их в один список:

²¹ Compass Reference // Compass-style.org. — http://compass-style.org/reference/compass/typography/vertical_rhythm/.

- сложная установка, ориентированная скорее на программистов, повышает порог вхождения для всех остальных. Однако в последнее время эта проблема решается появлением средств для установки с графическим интерфейсом;
- встроенные миксины, как в самих препроцессорах, так и в библиотеках зачастую содержат некорректный или устаревший код. Это касается прежде всего миксинов, призванных решать проблемы с префиксными свойствами. Альтернативой им могут быть только вручную создаваемые миксины;
- избыточная вложенность и чрезмерное использование миксинов могут сделать конечный код CSS тяжеловесным. Замена миксинов без аргументов на директиву `@extend` и контроль за вложенностью²² эту проблему решают;
- код препроцессора достаточно сложно отлаживать, поскольку при генерации конечного файла все упоминания о его специфических особенностях исчезают.

Препроцессоры — это CSS будущего?

Если рассматривать появление и развитие препроцессоров в русле развития веб-стандартов, можно отметить, что большинство их функций является частью общей тенденции к упрощению синтаксиса технологий и в то же время лучшей их приспособленности к практическим нуждам веб-дизайна. В этом плане в один ряд с препроцессорами CSS можно поставить другие метаязыки, получившие популярность в последнее время, — препроцессоры HTML (HAML, Jade, Slim), языки, транслируемые в Javascript (CoffeeScript) или библиотеки, являющиеся надстройкой над ним (jQuery, MooTools). Вращаясь в тех же кругах Всемирной паутины, что и официальные рабочие группы, развивающие клиентские технологии в Вебе, разработчики препроцессоров задают весьма схожий с последними вектор развития, однако позволяют себе меньшую осторожность в имплементации и утверждении новых функций.

Показательно, что попытки "улучшить" CSS здесь и сейчас осуществляются не только в рамках создания препроцессоров, но и в другой стремительно развивающейся сфере — экосистеме Javascript-библиотек, предлагающей свои варианты решения проблем с префиксами, манипуляцией цветами и др. Точно так же, как некоторые функции, ранее реализовывавшиеся при помощи скриптов, постепенно входят в практику CSS-оформления (трансформации, анимации, уголки и т.д.), так и определенные новшества, "обкатываемые" в препроцессорах, становятся модулями традиционного CSS²³. Некоторые из них на данный момент

²² Так, официальный блог SASS The Sass Way рекомендует использовать не больше 4 уровней вложенности — см. Ricalde M. Nested Selectors: The Inception Rule // The Sass Way. 2011. — 20 Nov. — <http://thesassway.com/beginner/the-inception-rule>.

²³ Atkins T.Jr. CSSOM, Vars, Mixins, Nesting, and Modules // Tab Completion. 2011. — 22 Jan. — <http://www.xanthir.com/blog/b49w0>.

находятся в статусе черновика и/или только готовятся к реализации в браузерах²⁴. На пути этого процесса, безусловно, стоят сложные и тяжеловесные бюрократические процедуры формирования и прогресса нормативных документов W3C. Однако, по мнению некоторых специалистов, вовсе не все новшества, введенные препроцессорами, заслуживают переноса в CSS — в первую очередь из-за возможных проблем с производительностью²⁵. Таким образом, присутствует мнение, что препроцессоры вовсе не являются временным явлением, призванным лишь предварить или обкатать нововведения будущих модулей CSS.

Обобщим основные направления, по которым препроцессоры улучшают работу со стилевым оформлением:

- автоматизация трудоемких процессов и методик, способствующих оптимизации технических качеств CSS-документов и облегчению работы со сложными визуальными эффектами;
- обеспечение лучшей читаемости и компактности кода;
- введение новых методик организации кода;
- повышение визуального единообразия веб-страницы и облегчение визуальной нюансировки.

Этот список свидетельствует, что к аудитории, использующей препроцессоры, вряд ли можно отнести только верстальщиков, работающих исключительно с кодом. Фундаментальные идеи, лежащие в основе препроцессоров, адресованы не только им, но и тем, кого называют веб-дизайнерами²⁶. Находятся и те, кто считает, что препроцессоры сближают верстку скорее с программированием, чем с дизайном²⁷. Как бы то ни было, попробуем обобщить направления нововведений, значимых в контексте дизайна:

- конкретизация констант оформления, в т.ч. оформительских приемов;
- легкость повторного использования и глобального изменения констант;
- выстраивание иерархии констант, задание зависимостей между ними и легкое управление этими зависимостями;
- минимизация отвлекающих от характеристик оформления компонентов кода (дублирующих, решающих проблемы технического характера и т.д.).

²⁴ Например, черновик переменных в CSS доступен по адресу <http://www.w3.org/TR/css-variables/>.

²⁵ См. комментарий разработчика интерфейсов компании "Яндекс" Р. Комарова по адресу http://habrahabr.ru/company/yandex/blog/169415/#comment_5875959.

²⁶ Будто в подтверждение этой мысли в 2013 г. в издательстве A Book Apart выходит книга известного специалиста по CSS Д. Седерхольма, озаглавленная SASS for Web Designers.

²⁷ Long J.W. The demise of plain CSS: Why Sass and languages like it will triumph // Wiseheart Design. 2010. — 18 Jan. — <http://wiseheartdesign.com/articles/2010/01/18/the-demise-of-css-why-sass-and-languages-like-it-will-triumph/>.

Можно сказать, что, даже несмотря на наличие корней в программировании, препроцессоры направлены на то, чтобы вернуть верстку в лоно творческого процесса, сделать ее из чисто технической процедуры процедурой, в ходе которой решаются креативные задачи. Это может заставить нас еще раз задуматься об трудности определения веб-дизайна как междисциплинарной по своей сути профессии.

Как мы видим, препроцессоры стали логичным и своевременным ответом на вызовы своего времени. Частично их можно считать пионерами каскадных таблиц стилей будущего, однако сложно сомневаться и в том, что самим препроцессорам уготован долгий путь самостоятельного развития.

Список литературы:

1. Мейер Э. CSS – каскадные таблицы стилей. Подробное руководство. С.-Пб.: Символ-Плюс, 2006.
2. Lie H.W., Bos B. Cascading Style Sheets – Designing for the Web. Addison-Wesley Professional, 2005.
3. Marcotte E. Responsive Web Design. A Book Apart, New York, 2011.
4. CSS-Tricks. – <http://css-tricks.com>.
5. HTTP Archive. – <http://httparchive.org>.
6. Smashing Magazine. – <http://smashingmagazine.com>.
7. The Sass Way. – <http://thesassway.com>.
8. World Wide Web Consortium. – <http://w3.org>.